

Analisi in Tempo Reale di Dati sui Voli

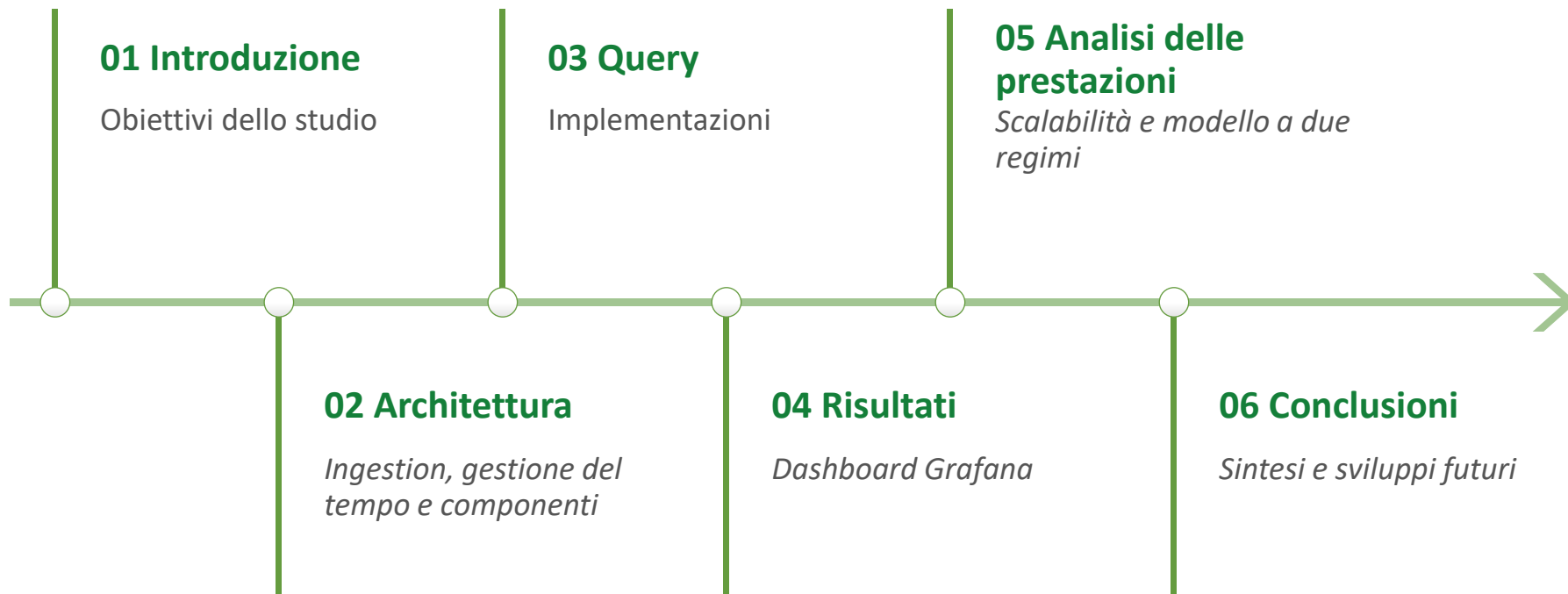
con Apache Flink

Pipeline di Stream Processing su Architettura Containerizzata

SABD A.A. 2025/26 | Prof. Cardellini & Nardelli

Valentina Jin

Ingegneria Informatica – Università degli Studi di Roma Tor Vergata



01 Introduzione

Il dominio



- **Stream processing** in tempo reale su dati di volo del **Bureau of Transportation Statistics** (BTS) degli Stati Uniti.
- **Stesso dataset di Project 1** ($\approx 2,2$ mln voli, Gennaio–Aprile 2025), ora elaborato come **flusso continuo di eventi** anziché in batch.

Obiettivi dello studio



Q1

Monitoraggio Compagnie
Aeree AA, DL, UA, WN

E1

Scalabilità Task Slot

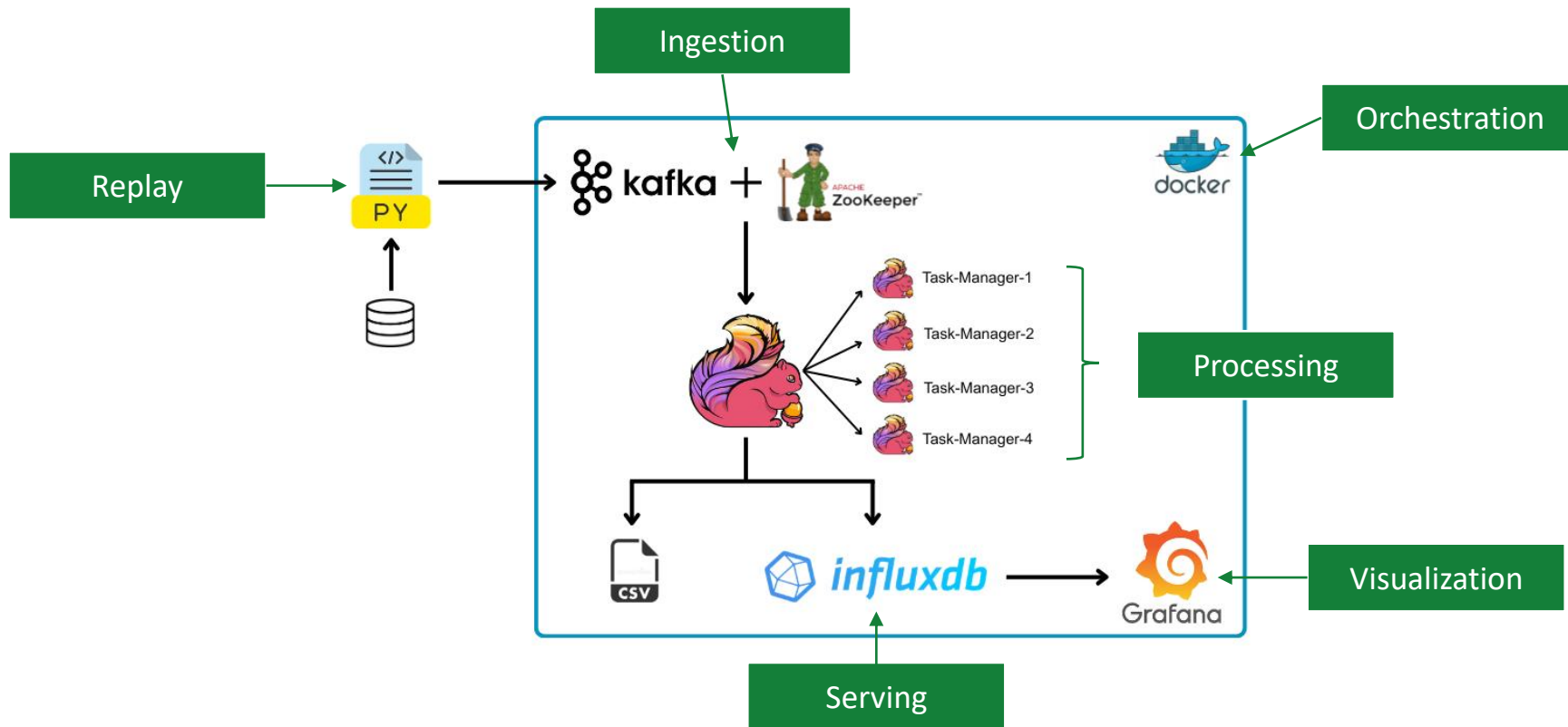
Q2

Ranking Top-10 Aeroporti per
Ritardi Significativi (> 30 min)

E2

Scalabilità TaskManager

02 Architettura

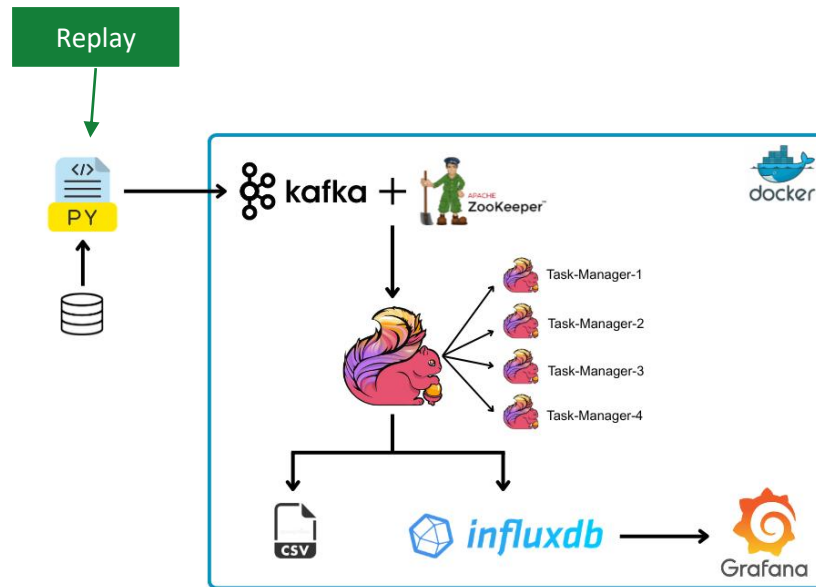


Replay

Producer (Python):

- legge i CSV mensili;
- calcola l'event time componendo YEAR, MONTH, DAY_OF_MONTH e CRS_DEP_TIME;
- ordina il dataset per l'event time crescente;
- effettua il replay verso Kafka con fattore di accelerazione configurabile (TIME_SCALE_FACTOR).

Heartbeat notturni: quando il gap tra due voli reali supera 5 minuti di event time, il producer inserisce messaggi fittizi ogni 10 minuti per mantenere il watermark in avanzamento; sono scartati da un filtro `isHeartbeat()` prima di qualunque aggregazione.



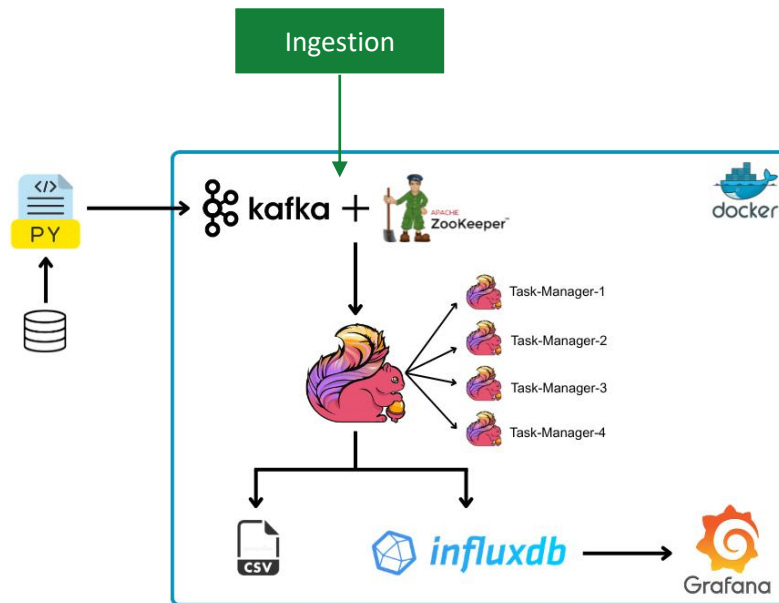
Ingestion

Apache Kafka: (broker singolo, coordinato da Zookeeper)

- riceve gli eventi pubblicati dal producer sul topic `flights`; disaccoppia temporalmente producer e motore di elaborazione.

Singola partizione: preserva l'ordine di consegna coerente con l'ordine di produzione, evitando margini di tolleranza al watermark più ampi.

`kafka_produce_time` incluso in ogni messaggio, usato per il calcolo della latenza end-to-end.

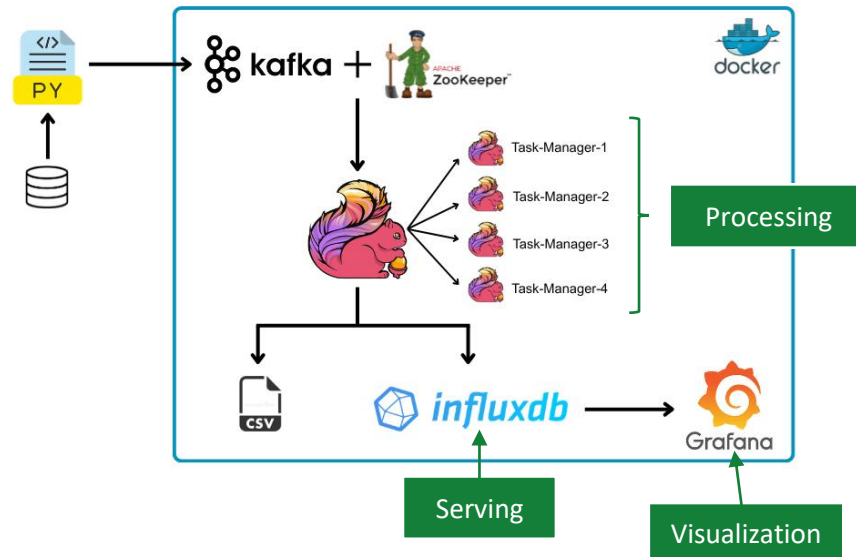


Processing & Serving & Visualization

Apache Flink 1.18 (Java 11): JobManager + numero variabile di TaskManager, watermark su event time, esegue le pipeline Q1 e Q2, scrive su CSV e InfluxDB.

InfluxDB: TSDB utilizzato come sink secondario per le metriche prodotte dalle query, alimentato in parallelo alla scrittura su file CSV.

Grafana: fornisce due dashboard, una per query, che interrogano InfluxDB per la visualizzazione in tempo reale dell'andamento delle statistiche prodotte da Flink.



Watermarking e Scelte di Design

Gestione del Watermark



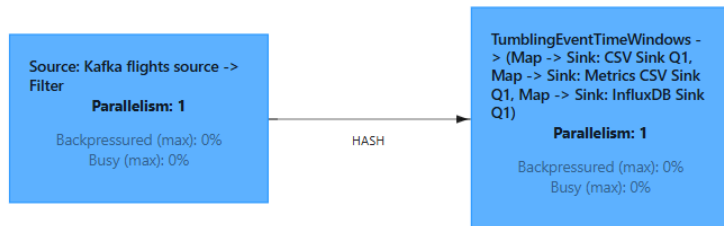
- **BoundedOutOfOrderness(5 min di event time):**
 - tollera il disordine residuo per via del batching del consumer Kafka.
- **withIdleness(10s di wall-clock):**
 - previene il blocco del watermark quando il parallelismo di job supera il numero di partizioni Kafka.

Scelte di Design



- **Producer legge e trasmette su Kafka solo i campi utili alle query:** per non appesantire il broker.
- **Event time:** le righe con CRS_DEP_TIME anomalo (fuori range 0000–2359) sono scartate a monte da producer.
- **Valori mancanti:** DEP_DELAY nullo è escluso dal calcolo delle medie (evita un bias verso il basso); CANCELLED e DIVERTED nulli sono trattati come 0 ("non cancellato/deviato"), per non perdere il volo dal conteggio.

03 Query



Obiettivo

- Per AA, DL, UA, WN: statistiche orarie su voli completati, cancellati e devianti — ritardo medio in partenza per voli non cancellati, tasso di cancellazione, tasso di partenze in ritardo (>15 min).

Pipeline

- KafkaSource** (watermark 5 min + idleness 10s)
- filter** compagnie target → **keyBy(airline)**
- TumblingEventTimeWindows(1h)**
- ProcessWindowFunction**: contatori + medie
- Sink** (parallelismo 1): CSV + InfluxDB

```
// — 2. Sorgente Kafka —
KafkaSource<FlightEvent> kafkaSource = FlinkSourceBuilder.buildKafkaSource(
    KAFKA_BROKER, KAFKA_TOPIC, KAFKA_GROUP_ID);

WatermarkStrategy<FlightEvent> watermarkStrategy =
    FlinkSourceBuilder.buildWatermarkStrategy()
        .withIdleness(Duration.ofSeconds(10));

DataStream<FlightEvent> flights = env
    .fromSource(kafkaSource, watermarkStrategy, "Kafka flights source");

// — 3. Pipeline Q1 —
DataStream<Q1WindowResult> q1Results = flights
    .filter(e -> e.getAirline() != null
        && TARGET_AIRLINES.contains(e.getAirline()))
    .keyBy(FlightEvent::getAirline)
    .window(TumblingEventTimeWindows.of(Time.hours(1)))
    .process(new Q1ProcessWindowFunction());
```

03 Query 2: Ranking Aeroporti per Ritardi Significativi



Obiettivo

- Tra i voli completati, i **Top-10 aeroporti di partenza** per ritardi significativi ($\text{dep_delay} > 30 \text{ min}$) con almeno 30 voli completati.

Tre granularità temporali

- SlidingEventTimeWindows(1h, slide 60min)** — equivalente a una tumbling window;
- SlidingEventTimeWindows(6h, slide 60min)** — ranking aggiornato ogni ora su finestra più ampia;
- GlobalWindows + ContinuousEventTimeTrigger(60min)** — ranking cumulativo dall'inizio dello stream.

Ogni windowAll è eseguito da Flink a parallelismo 1, indipendentemente dal parallelismo del job

03 Query 2: Ranking Aeroporti per Ritardi Significativi

// — 2. Sorgente Kafka —

```
KafkaSource<FlightEvent> kafkaSource = FlinkSourceBuilder.buildKafkaSource(  
    KAFKA_BROKER, KAFKA_TOPIC, "flink-q2-group");
```

```
WatermarkStrategy<FlightEvent> watermarkStrategy =  
    FlinkSourceBuilder.buildWatermarkStrategy()  
        .withIdleness(Duration.ofSeconds(10));
```

```
DataStream<FlightEvent> flights = env  
    .fromSource(kafkaSource, watermarkStrategy, "Kafka flights source Q2");
```

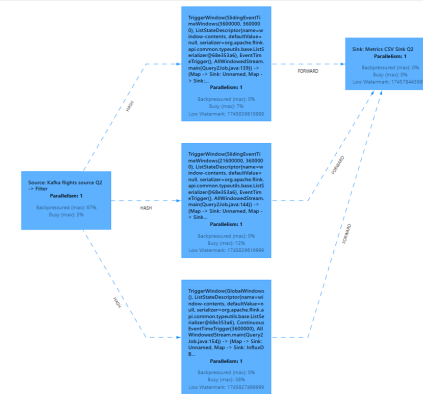
// — 3. Pre-filtro: scarta heartbeat e voli non completati —

```
DataStream<FlightEvent> completedFlights = flights  
    .filter(e -> !e.isHeartbeat() && e.isCompleted() && e.getOriginAirportId() != null);
```

// — 4. Finestra 1h sliding (slide 60min) —

```
DataStream<Q2RankedEntry> ranked1h = completedFlights  
    .windowAll(SlidingEventTimeWindows.of(Time.hours(1), Time.minutes(60)))  
    .process(new Q2SlidingRankingFunction());
```

Q2SlidingRankingFunction/ Q2GlobalRankingFunction: tramite `HashMap<Integer, Q2AirportStats>` ricostruita a ogni trigger: conteggio voli, ritardi severi, somma/massimo del ritardo, top-20 voli più ritardati



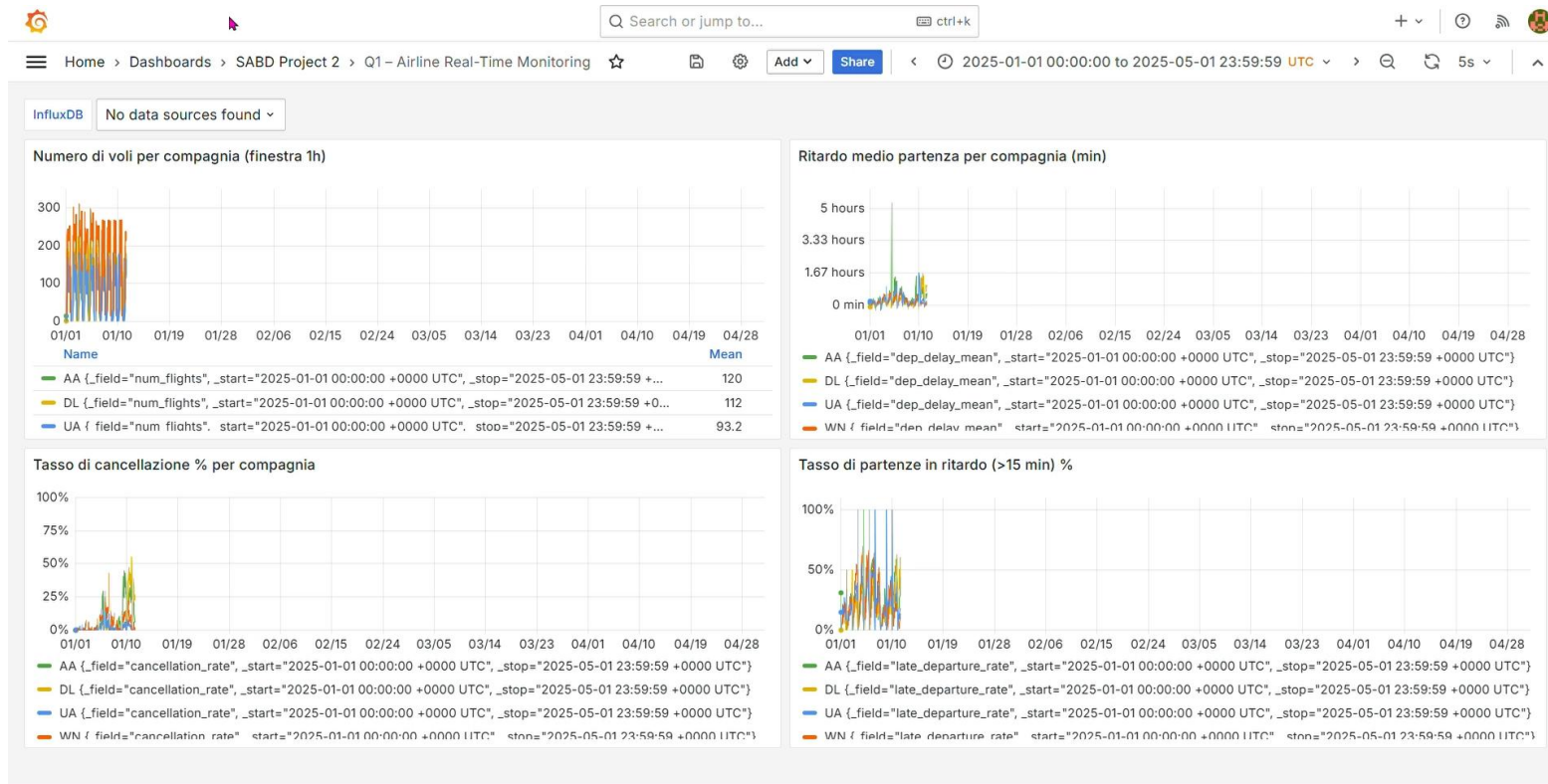
// — 5. Finestra 6h sliding (slide 60min) —

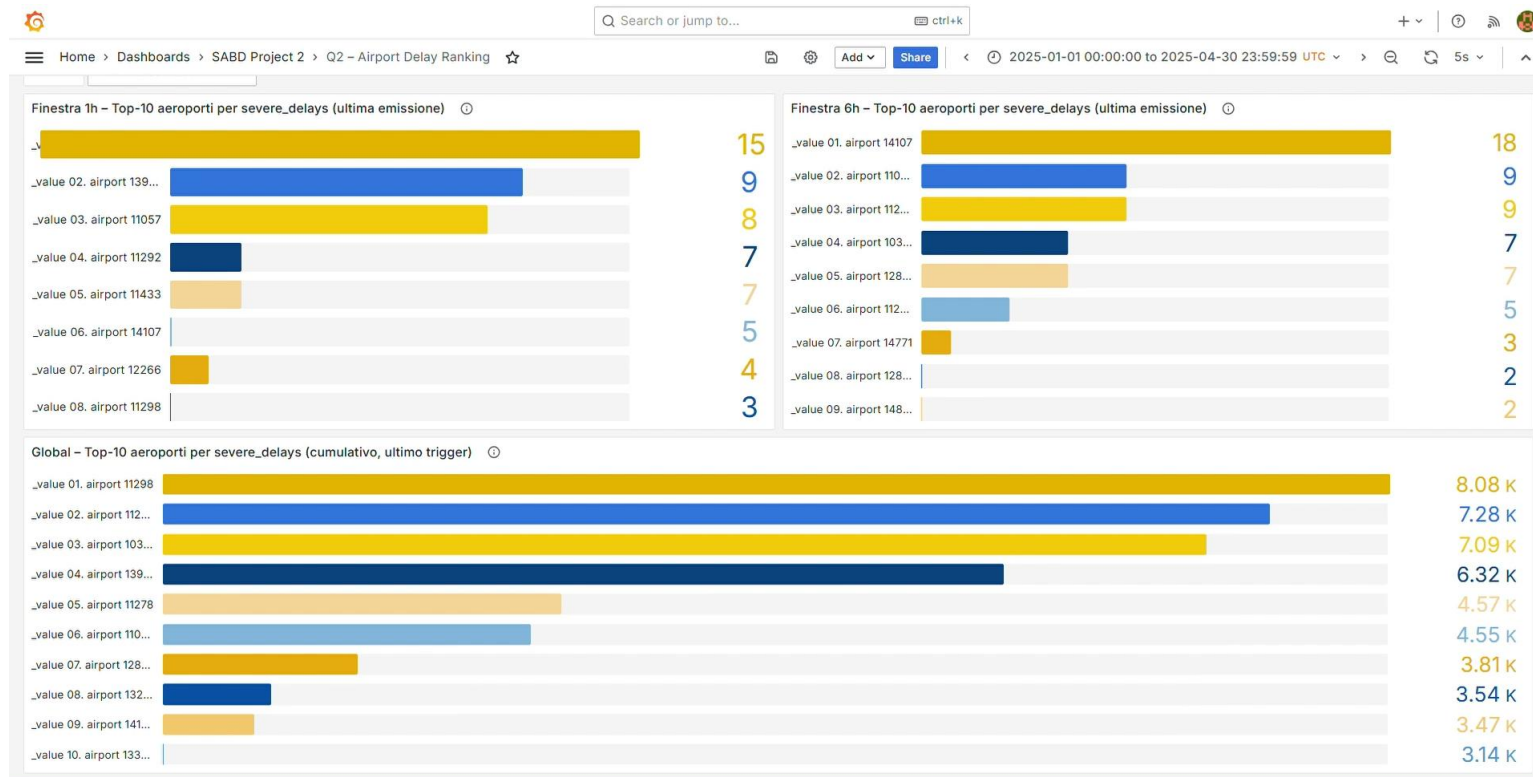
```
DataStream<Q2RankedEntry> ranked6h = completedFlights  
    .windowAll(SlidingEventTimeWindows.of(Time.hours(6),  
        Time.minutes(60)))  
    .process(new Q2SlidingRankingFunction());
```

// — 6. Finestra global (trigger ogni 60min di event time) —

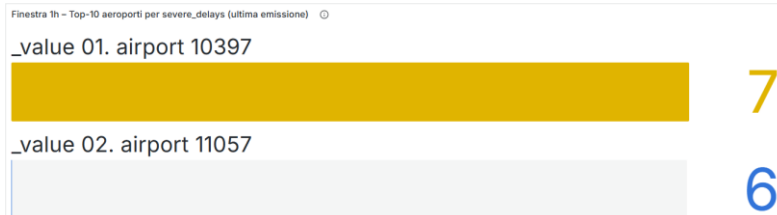
```
DataStream<Q2RankedEntry> rankedGlobal = completedFlights  
    .windowAll(GlobalWindows.create())  
    .trigger(ContinuousEventTimeTrigger.of(Time.minutes(60)))  
    .process(new Q2GlobalRankingFunction());
```

04 Risultati

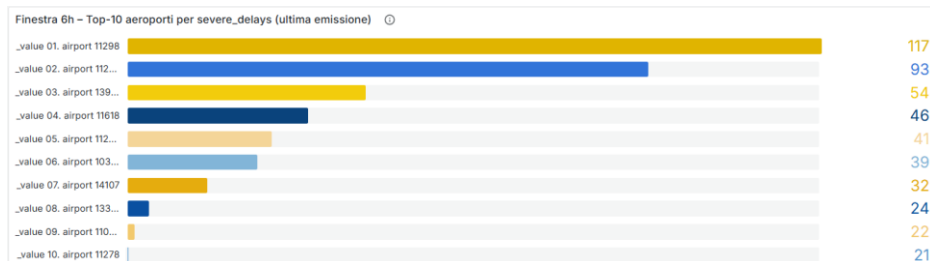




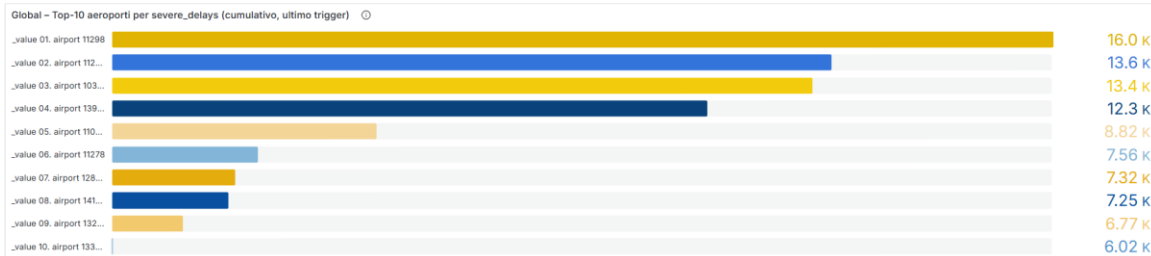
Finestra 1h



Finestra 6h



Finestra globale cumulativa



05 Analisi delle Prestazioni

Metodologia

Hardware

Intel Core i5-1135G7 (4 core fisici, 8 logici), 16GB RAM, WSL2 + Docker Compose, deploy locale su singolo host

Doppio Canale

- **Canale esterno** (latenza end-to-end da kafka_produce_time, throughput da voli/durata)
- **Canale interno** (REST API Flink, EWMA, numRecordOutPerSecond)
- Scarto sistematico 5–10%, per validare reciprocamente throughput.

Esperimenti

TIME_SCALE_FACTOR = 86400, 3 ripetizioni:

- (1) parallelismo del job su 1 TaskManager multi-slot;
- (2) numero di TaskManager con 1 slot ciascuno ($N = 1, 2, 4$)

Metrica Busy%

Percentuale di tempo in cui un operatore è occupato: la metrica più affidabile per isolare l'effetto del parallelismo

Modello a Due Regimi (Bottleneck): assumendo che il carico è distribuito uniformemente su tutti i p slot attivi

Utilizzo teorico di ciascuno slot

$$\rho(p) = \lambda / (p \cdot \mu_1)$$

- λ : throughput sostenibile dal producer Python.
- p : numero di task slot attivi.
- μ_1 : capacità di elaborazione di un singolo slot Flink.



Parallelismo critico p_c

$$\rho(p_c) = 1 \quad p_c = \lambda / \mu_1$$



Throughput realmente con TSF

$$\lambda(TSF) = \min(\lambda_{target}(TSF), \mu_p)$$

- $\lambda_{target}(TSF)$: il tasso di emissione teorico.
- μ_p : il tasso di emissione massima effettivamente sostenibile dal producer Python.

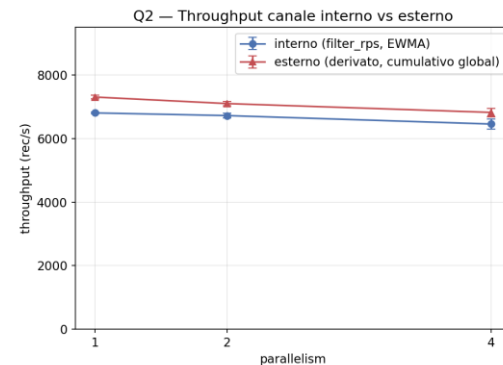
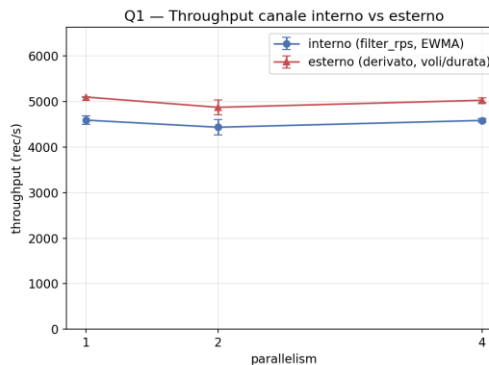


Poiché $\lambda_{target}(TSF)$ cresce linearmente con TSF, esiste un **valore critico TSF*** oltre il quale il producer satura:

- **TSF < TSF***: il sistema è vincolato dalla scelta del fattore di accelerazione TSF;
- **TSF ≥ TSF***: regime producer-bound, il producer Python (single-thread/GIL) satura;
- **TIME_SCALE_FACTOR = 86400** supera ampiamente TSF* ($\cong 23.722$): l'intera valutazione sperimentale opera in regime producer-bound.

E1: Scalabilità a livello di Task Slot

Query	TS	Throughput	Latenza (ms)	Busy%
Q1	1	5099	167,4	9,17%
Q1	2	4874	169,5	4,10%
Q1	4	5029	171,7	4,23%
Q2 (global)	1	7304	1414	58,8%
Q2 (global)	2	7099	843	53,8%
Q2 (global)	4	6821	801	52,7%

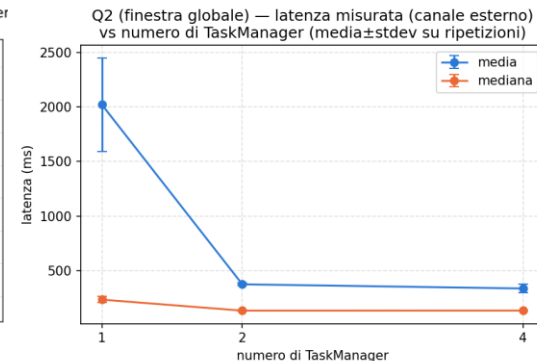
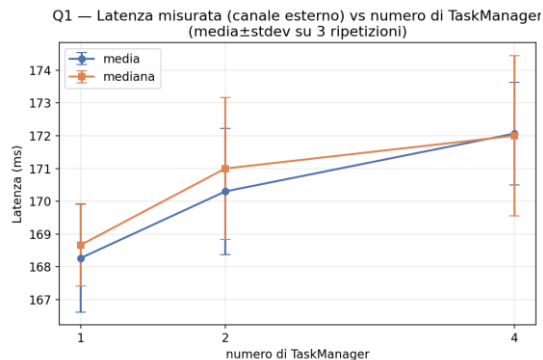


Parallelismo del job su singolo TaskManager multi-slot (N = 1, 2, 4)

- Busy% di Flink è basso (Q1).
- **Busy% elevato per Q2** -> costo computazionale per trigger cresce $O(n)$ con l'accumulo di dati.
- Calo di Busy% di Q2 riflette la parallelizzazione degli operatori a monte (source, filter) e la minore contesa di scheduling.
- **Vincoli strutturali aggiuntivi:** keyBy(airline) su 4 chiavi (Q1); windowAll intrinsecamente seriale a parallelismo 1 (Q2).

E2: Scalabilità a livello di TaskManager

Query	TM	Throughput	Latenza med.	Busy%
Q1	1	5093	168,7	9,27%
Q1	2	4893	171,0	4,14%
Q1	4	4753	172,0	4,07%
Q2 (global)	1	7130,16	233	61,5%
Q2 (global)	2	6985,94	132	49,2%
Q2 (global)	4	6535,76	132,5	49,6%

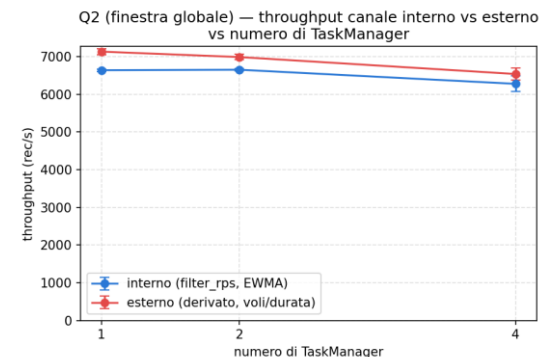
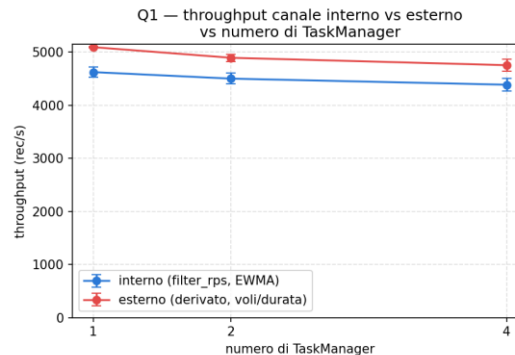


Numero di TaskManager con un solo slot ciascuno (N = 1, 2, 4)

- **Q2, media vs mediana:** a TM=1 la media (2022,5 ms) è distorta da pochi trigger tardivi molto più costosi (mediana 233 ms) — la mediana è la statistica di riferimento per la finestra globale.
- **Vincoli strutturali aggiuntivi:** keyBy(airline) su 4 chiavi (Q1); windowAll intrinsecamente seriale a parallelismo 1 (Q2).

E2: Scalabilità a livello di TaskManager

Query	TM	Throughput	Latenza med.	Busy%
Q1	1	5093	168,7	9,27%
Q1	2	4893	171,0	4,14%
Q1	4	4753	172,0	4,07%
Q2 (global)	1	7130,16	233	61,5%
Q2 (global)	2	6985,94	132	49,2%
Q2 (global)	4	6535,76	132,5	49,6%



Numero di TaskManager con un solo slot ciascuno (N = 1, 2, 4)

- **Throughput:** cala del 7–8% con più TaskManager, per l’overhead di rete/JVM tra processi distinti rispetto a slot condivisi sulla stessa JVM.
- Lo scarto tra due canale si mantiene nell’ordine del 5–10%, confermando la coerenza delle due misurazioni indipendentemente su tutta la scala di parallelismo testata.

06 Conclusioni

1. **Risultato chiave:** il parallelismo Flink non è mai stato il fattore limitante — sistema producer-bound su tutta la scala testata.
2. **Vincoli di design specifici:** partizionamento su 4 chiavi (Q1), operatore windowAll seriale (Q2) — limiti teorici ulteriori non ancora osservabili nell'intervallo testato.
3. **Limite principale:** singolo host con risorse condivise tra i container, non isola con certezza il parallelismo Flink dalla contesa di Sistema.

Sviluppi futuri:

1. Cluster distribuito multi-nodo;
2. Estensione a Q3 con checkpointing/fault tolerance;
3. Rimozione del collo di bottiglia lato producer;
4. Accumulatore incrementale per la finestra globale di Q2.

Grazie per l'attenzione!

SABD A.A. 2025/26 | Prof. Cardellini & Nardelli

Valentina Jin – valentina.jin@students.uniroma2.eu